Phishing Website Detection using Machine Learning

Machine Learning Project Report

Submitted in fulfillment of the requirements

for the degree of

Bachelor of Technology

in

Information Technology

TY B.TECH INFT SEM V

by

Kinage Parth (24101B0036)
Boddu Ronak (24101B0044)
Pokharkar Vedangi (24101B0051)

Project Guide

Dr. Kanchan Dhuri



Department of Information Technology
Vidyalankar Institute of Technology
Wadala (E), Mumbai – 400037
(Affiliated to University of Mumbai)

2026-2027



Certificate

This is to certify that the project entitled

“Phishing Website Detection using Machine Learning”

is a bonafide work done by

Kinage Parth (24101B0036)

Boddu Ronak (24101B0044)

Pokharkar Vedangi (24101B0051)

and is submitted in fulfillment of the requirement for

TY B.TECH INFT SEM V

to the

Vidyalankar Institute of Technology, Wadala, Mumbai.

Project Guide
Dr. Kanchan Dhuri

Abstract

Phishing websites imitate trusted organisations to steal passwords and payment details, and they appear faster than blocklists can track them. This project develops and deploys a machine learning system that classifies a website as phishing or legitimate. Using the UCI Phishing Websites dataset (11,055 websites, 30 features), we train and tune four classifiers, namely Decision Tree, Random Forest, Support Vector Machine and XGBoost, with 5-fold cross-validation on two feature sets: all 30 features, and the 21 features that a program can compute from a live URL and its HTML. On the held-out test set, Random Forest reaches 97.69% accuracy and XGBoost 97.60% with the best recall and ROC-AUC (0.9969); the deployable feature set reaches 94.5–94.8%. Exploratory analysis revealed that 47% of the rows are duplicates; removing them lowers the F1-score by 2–4 points, a more realistic estimate. Permutation importance and SHAP agree that valid HTTPS and the destination of a page's links are the strongest signals, and that the deployable model's gap comes from missing site-reputation features. The best deployable model, XGBoost, is served by a FastAPI service with a web page, a Chrome extension and a public website, and explains every decision with SHAP values. A modern, cleanly built phishing page is still missed, which highlights the need for fresher training data.

Keywords: Phishing detection, Machine learning, XGBoost, Random Forest, SHAP, Web security

Contents

Abstract iii

List of Figures vi

List of Tables vii

Abbreviations viii

Chapter 1 Introduction 1
1.1 Background 1
1.2 Motivation 2
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Scope of the Work 3
1.6 Organization of the Report 3

Chapter 2 Literature Survey 4
2.1 Review of Existing Work 4
2.2 Summary of Literature 4
2.3 Research Gaps 5

Chapter 3 Proposed Methodology 6
3.1 System Architecture 6
3.2 Dataset Description 6
3.2.1 Exploratory Data Analysis (EDA) Results 7
3.3 Algorithm / Model Description 9
3.3.1 Models Used 10
3.3.2 Model Development and Hyper-parameter Tuning 10
3.3.3 Feature Importance Methods 11
3.3.4 Performance Metrics 11
3.4 Tools and Technologies Used 12

Chapter 4 Results and Discussion 13
4.1 Experimental Setup 13
4.2 Results 13
4.2.1 Model Comparison on the Full Feature Set 13
4.2.2 Model Comparison on the Lite (Deployable) Feature Set 14
4.2.3 Sensitivity to Duplicate Rows 15
4.2.4 Feature Importance 16
4.2.5 Sample Results of the Deployed Application 20
4.3 Comparison with Existing Methods 22
4.4 Deployment Details 23
4.5 Link of GitHub Repository 23

Chapter 5 Conclusion and Future Scope 25
5.1 Conclusion 25
5.2 Future Scope 25

Chapter 6 List of Publications 26
6.1 List of Published Papers 26
6.2 Details of Participation in Project Competitions 26
References 27

List of Figures

Figure 3.1: Block diagram of the proposed system 6
Figure 3.2: Class distribution and duplicate rows 8
Figure 3.3: Phishing rate for each value of eight key features 8
Figure 3.4: Correlation of each feature with the phishing label (bold: not in the lite set) 9
Figure 4.1: ROC curves on the test set (full feature set) 14
Figure 4.2: Confusion matrices on the test set (full feature set) 14
Figure 4.3: ROC curves on the test set (lite feature set) 15
Figure 4.4: Permutation importance on the test set (top 15 features per model) 17
Figure 4.5: SHAP summary for the Random Forest (500 test websites) 18
Figure 4.6: Agreement between importance methods (Spearman correlation) 19
Figure 4.7: Ablation: Random Forest on its top-k features 20
Figure 4.8: Share of the full-vs-lite gap recovered by each excluded feature 20
Figure 4.9: Sample output: a legitimate website checked on the public deployment 21
Figure 4.10: Sample output: the fake phishing test page is detected (99.5%) 21
Figure 4.11: Sample output: a modern, clean fake phishing page is missed (limitation) 22

List of Tables

Table 2.1: Summary of literature survey 5
Table 3.1: Dataset description..... 6
Table 3.2: Feature sets used..... 7
Table 3.3: Hyper-parameter grids and best configurations (5-fold CV) 10
Table 4.1: Test-set performance on the full feature set (30 features, 2,211 test websites) 13
Table 4.2: Test-set performance on the lite feature set (21 features)..... 14
Table 4.3: Accuracy and F1-score on the standard and de-duplicated splits..... 15
Table 4.4: Performance comparison of the proposed method with existing methods 22

Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
AUC	Area Under the Curve
CNN	Convolutional Neural Network
CV	Cross-Validation
DNS	Domain Name System
DT	Decision Tree
EDA	Exploratory Data Analysis
FN / FP	False Negative / False Positive
HTML	HyperText Markup Language
HTTPS	HyperText Transfer Protocol Secure
ML	Machine Learning
RBF	Radial Basis Function
RF	Random Forest
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
SSRF	Server-Side Request Forgery
SVM	Support Vector Machine
TLS / SSL	Transport Layer Security / Secure Sockets Layer
TN / TP	True Negative / True Positive
UCI	University of California, Irvine (ML repository)
URL	Uniform Resource Locator
WHOIS	Domain registration lookup protocol
XGBoost	eXtreme Gradient Boosting

Chapter 1
Introduction

Phishing is a form of online fraud in which an attacker builds a website that imitates a trusted organisation, such as a bank, an e-mail provider or an online shop, in order to steal passwords, card numbers and other personal data. Phishing pages are cheap to create and are usually online for only a short time, so lists of known bad sites are always one step behind the attackers.

This project builds a machine learning (ML) system that decides whether a website is **phishing** or **legitimate** from measurable properties of its address and page. Four supervised classifiers are trained and compared on a public benchmark, the most important warning signs are identified with several feature-importance methods, and the best deployable model is turned into a working application: a web service, a web page and a Chrome extension that check any URL and explain the verdict.

1.1 Background

Mohammad, Thabtah and McCluskey [1] proposed a set of 30 phishing indicators and published the UCI Phishing Websites dataset [2]. The indicators fall into four groups: address-bar features (for example an IP address used as the host, a very long URL, an '@' symbol or a '-' in the domain), HTML and JavaScript features (where links, forms, scripts and images point to), domain features (domain age, DNS record, registration length) and reputation features (web traffic rank, PageRank, Google index, number of backlinks). Each indicator is coded as -1 (phishing-like), 0 (suspicious) or 1 (legitimate-like).

Detecting phishing from these indicators is a supervised binary classification problem. Tree-based ensembles such as Random Forest [6] and gradient-boosted trees (XGBoost) [7] usually perform well on such tabular data, while kernel methods such as the Support Vector Machine (SVM) [9] are a strong classical baseline. Explanation methods such as SHAP [8] and permutation importance [11] show which features drive the predictions.

1.2 Motivation

Many published models report very high accuracy on this dataset, but they are rarely turned into a tool that can check a real website. Several of the original features can no longer be computed: Alexa traffic rank and Google PageRank have been discontinued, and WHOIS and search-engine lookups are slow or rate-limited. We wanted to find out how much accuracy remains when a model uses only features that can be computed live, which features matter most, and whether the high reported accuracy survives a stricter evaluation. A working, explainable demo also makes the work useful beyond a single table of numbers.

1.3 Problem Statement

Given a website URL, classify the website as phishing or legitimate. The model is trained on the 30 pre-computed features of the UCI Phishing Websites dataset; for deployment, it must work from features that a program can compute from the URL and its static HTML within a few seconds, and it must explain each decision. Missing a phishing site is costlier than a false alarm, so recall and F1-score on the phishing class are the key metrics.

1.4 Objectives

The objectives of this project are:

- To study the UCI Phishing Websites dataset, its features and its quality issues (exploratory data analysis).
- To design and implement a reproducible training pipeline for Decision Tree, Random Forest, SVM and XGBoost classifiers on two feature sets: all 30 features (full) and the 21 features computable live (lite).
- To evaluate the performance of these models with cross-validation and a held-out test set, and compare it with existing methods.
- To identify the most important features with built-in, permutation and SHAP importance, and to explain the accuracy gap between the full and lite feature sets.
- To deploy the best lite model as a web service with a web page and a Chrome extension that give an explained verdict for any URL.

1.5 Scope of the Work

The work covers the 11,055-website UCI dataset, four classical ML models, CPU-only training on a laptop, and a live demo that analyses the static HTML of a page. It does not cover deep learning on raw page content, screenshots or e-mails, JavaScript-rendered pages, or a large-scale evaluation on today's phishing websites. The online version is a course demonstration, not a hardened security product.

1.6 Organization of the Report

Chapter 2 reviews existing work and identifies research gaps. Chapter 3 describes the proposed system: its architecture, the dataset with exploratory data analysis, the models, the training procedure and the evaluation metrics. Chapter 4 presents the experimental setup, the results, the feature-importance analysis, a comparison with existing methods and the deployment. Chapter 5 concludes and suggests future work, and Chapter 6 lists publications.

Chapter 2
Literature Survey

This chapter reviews work on ML-based phishing website detection that uses the same benchmark data, so that results can be compared.

2.1 Review of Existing Work

Mohammad, Thabtah and McCluskey [1] introduced the 30 phishing features used in this project and classified websites with a self-structuring neural network. Their dataset, published in the UCI repository [2], has since become a standard benchmark.

Awasthi and Goel [3] (2022) compared six base classifiers (logistic regression, naive Bayes, decision tree, SVM, k-nearest neighbours, linear discriminant analysis) with six ensembles (bagging, AdaBoost, gradient boosting, voting, Extra Trees and XGBoost) on the same 11,055-instance dataset, which they obtained from Kaggle. Extra Trees performed best with 98.73% accuracy and XGBoost reached 98.37%. These best results were obtained without cross-validation, using all 30 features.

Almujahid, Haq and Alshehri [4] (2024) compared logistic regression, k-nearest neighbours, decision tree, random forest, SVM, XGBoost, a convolutional neural network (CNN) and a dense neural network on two datasets, with feature selection (SelectKBest) and GridSearchCV tuning. On the UCI dataset, the tuned CNN achieved 0.99 accuracy, 0.99 precision, 0.98 recall and 0.99 F1-score. The authors note that the CNN was not interpreted and that real-world deployment was not evaluated.

Newaz, Haq and Ahmed [5] (2024) built a stacking ensemble: recursive feature elimination, ten base classifiers combined by a greedy search, and a multi-layer perceptron as the meta-classifier, trained with 10-fold cross-validation on four datasets. On the UCI dataset they report 97.49% accuracy, 97.11% precision, 98.43% recall and 97.76% F1-score, with Random Forest (97.28%) and XGBoost (97.29%) as the strongest single classifiers. The base learners were not tuned.

2.2 Summary of Literature

Table 2.1 summarizes the literature reviewed.

Table 2.1: Summary of literature survey

Sr. No.	Author(s) & Year	Method / Technique	Dataset	Result	Limitation
1	Awasthi & Goel, 2022 [3]	6 base + 6 ensemble classifiers; 10-fold CV for ensembles	UCI/Kaggle, 11,055 sites, 30 features	Extra Trees: 98.73% accuracy	Best results without CV; all 30 features; no deployment
2	Almujahid et al., 2024 [4]	LR, KNN, DT, RF, SVM, XGBoost, CNN, DNN; SelectKBest, GridSearchCV	Mendeley (10,000) and UCI (11,055)	CNN: 0.99 accuracy, 0.98 recall on UCI	No interpretability; no real-world deployment
3	Newaz et al., 2024 [5]	RFECV + 10 base learners, stacking with MLP meta-classifier	UCI + 3 Mendeley datasets	97.49% accuracy, 97.76% F1 on UCI	Untuned base learners; preprint; no live use

2.3 Research Gaps

The reviewed work leaves four gaps that this project addresses:

- **Evaluation on duplicated data.** Almost half of the dataset's rows are exact duplicates (Section 3.2.1), yet the reviewed studies evaluate on random splits without checking how duplicates inflate the scores.
- **Deployability.** The models use features (traffic rank, PageRank, Google index, backlinks, WHOIS) that cannot be computed for a new URL today, and none of the studies deploys a model that checks live websites.
- **Interpretability.** Few studies explain which features drive the predictions or whether different importance methods agree.
- **Full-vs-deployable gap.** No study measures how much accuracy is lost when only live-computable features are used, or which missing features cause the loss.

Chapter 3

Proposed Methodology

The proposed system has two parts: an offline pipeline that prepares the data, trains, tunes, evaluates and interprets four classifiers on two feature sets, and an online service that computes the deployable features for a live URL and returns an explained prediction.

3.1 System Architecture

The block diagram of the proposed system is shown in Figure 3.1. In the offline pipeline (A), the dataset is validated and split, the models are tuned with cross-validation, evaluated once on the test set and analysed with feature-importance methods. The best lite model is saved. In the online part (B), a client sends a URL to the FastAPI service, the feature extractor computes the 21 lite features, the XGBoost model returns a phishing probability, and SHAP values give the top reasons.

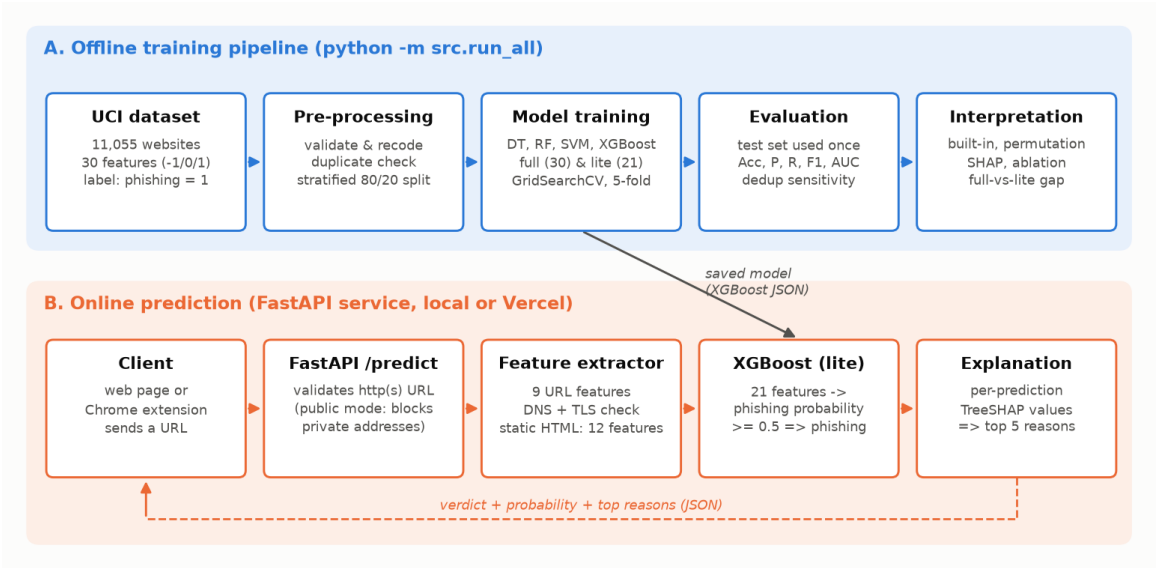


Figure 3.1: Block diagram of the proposed system

3.2 Dataset Description

The details of the dataset used are given in Table 3.1.

Table 3.1: Dataset description

Parameter	Details
Name of dataset	Phishing Websites (Mohammad, Thabtah and McCluskey)

Parameter	Details
Source	UCI Machine Learning Repository, dataset id 327 [2]
Number of samples	11,055 websites (5,849 unique rows)
Number of features	30, each coded -1 / 0 / 1 (no missing values)
Number of classes	2: phishing 4,898 (44.3%), legitimate 6,157
Train / Test split	Stratified 80 / 20, random_state = 42: 8,844 training, 2,211 test websites

The label is mapped to **phishing = 1** (the positive class) and legitimate = 0. The two feature sets are listed in Table 3.2. The lite set keeps the features that the extractor can compute from a URL and its static HTML; the nine excluded features need WHOIS, discontinued services, search-engine queries, blacklists or JavaScript execution.

Table 3.2: Feature sets used

Feature set	Features
URL features (9, lite)	having_ip_address, url_length, shortining_service, having_at_symbol, double_slash_redirecting, prefix_suffix, having_sub_domain, port, https_token
Page / network features (12, lite)	sslfinal_state, favicon, request_url, url_of_anchor, links_in_tags, sfh, submitting_to_email, redirect, on_mouseover, rightclick, iframe, dnsrecord
Excluded from lite (9)	domain_registration_length, abnormal_url, popupwindow, age_of_domain, web_traffic, page_rank, google_index, links_pointing_to_page, statistical_report

3.2.1 Exploratory Data Analysis (EDA) Results

The classes are moderately balanced: 44.3% of the websites are phishing (Figure 3.2). The most important quality issue is duplication: 5,206 of the 11,055 rows (47%) are exact copies of another row, and 64 feature vectors occur with both labels. Because a random split can put a row in the training set and its copy in the test set, the test scores can be optimistic; Section 4.2 measures this effect.

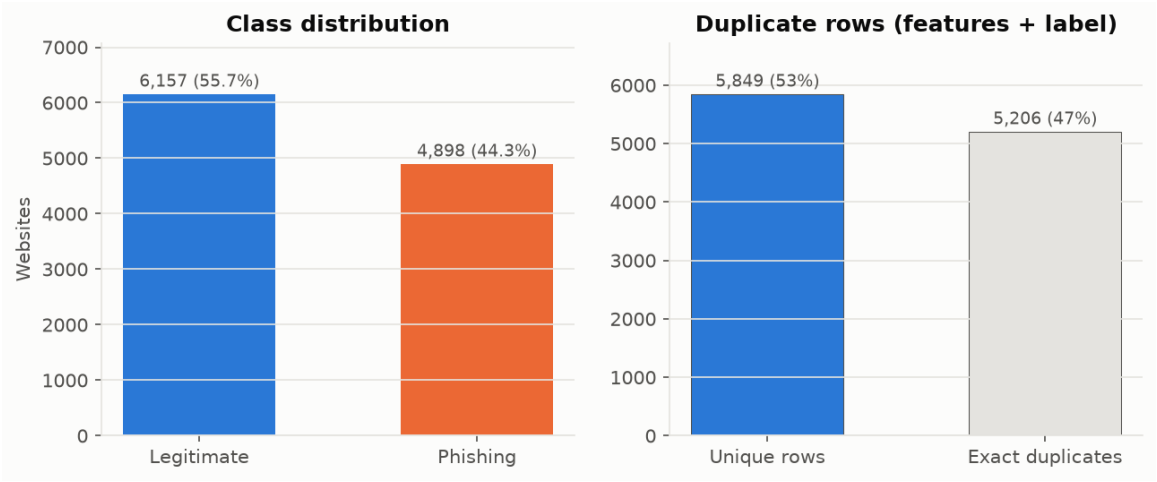


Figure 3.2: Class distribution and duplicate rows

Figure 3.3 shows the share of phishing websites for each value of eight key features. Websites with *url_of_anchor* = -1 (most links point nowhere or to other domains) are 99% phishing, and websites without valid HTTPS (*sslfinal_state* = -1 or 0) are mostly phishing, while a valid certificate (+1) means only 11% phishing. Figure 3.4 ranks all features by their correlation with the label: *sslfinal_state* (r = -0.72) and *url_of_anchor* (r = -0.69) are by far the strongest.

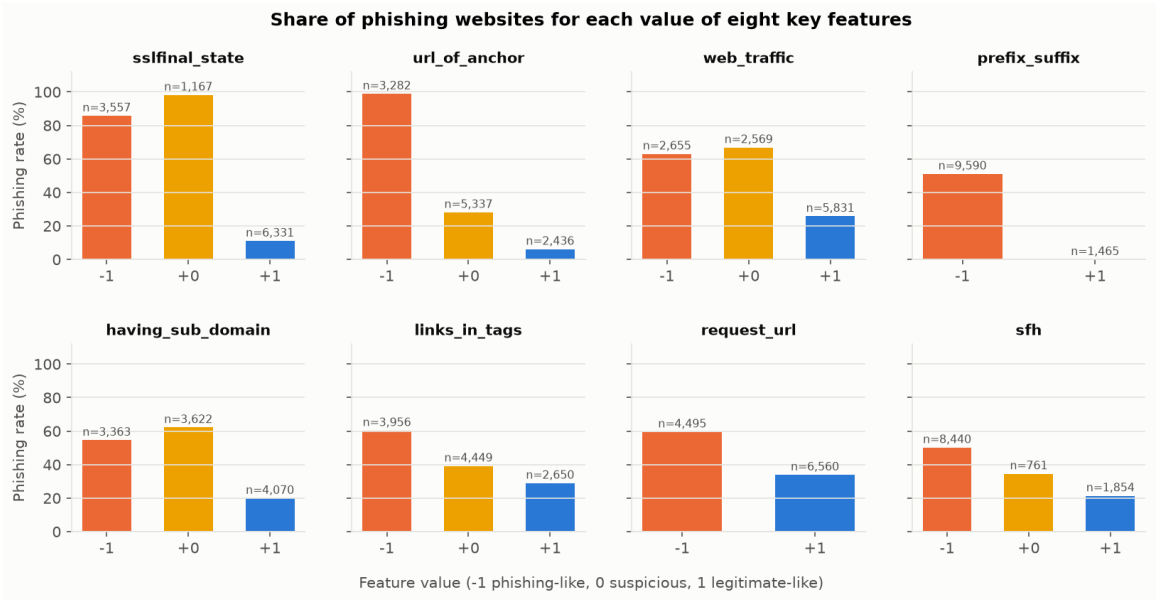


Figure 3.3: Phishing rate for each value of eight key features

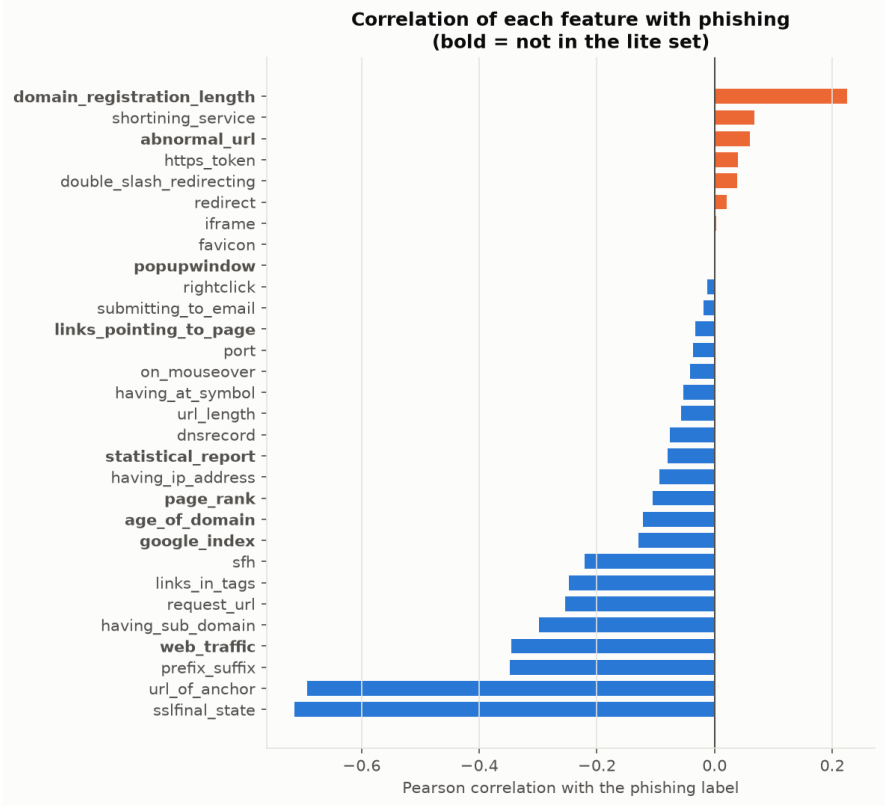


Figure 3.4: Correlation of each feature with the phishing label (bold: not in the lite set)

EDA also showed that the dataset's coding does not always follow its documentation. For example, *prefix_suffix* = -1 ("a '-' in the domain") occurs in 87% of all rows, and for five features (such as *shortining_service* and *domain_registration_length*) the documented phishing-like value is actually less common among phishing sites. This matters for the live demo, whose extractor follows the documented rules.

3.3 Algorithm / Model Description

The workflow is implemented in Python (src/ folder) and runs with one command (python -m src.run_all):

1. Load the dataset (ucimlrepo, with an ARFF download as fallback), validate the columns and values, map the label to phishing = 1 and summarise it (class balance, value counts, duplicates).
2. Split the data into stratified 80% training and 20% test sets with a fixed seed. The test set is used only once, for the final evaluation.
3. For each feature set (full, lite) and each model, tune the hyper-parameters with GridSearchCV and 5-fold stratified cross-validation on the training set, scoring the F1-score of the phishing class.

4. Refit the best configuration on the whole training set, measure training and single-prediction time, and save the model.
5. Evaluate every model on the test set and repeat the experiment on de-duplicated data.
6. Analyse feature importance and select the lite model with the best CV F1-score for deployment.

3.3.1 Models Used

- **Decision Tree:** a single tree of if-then splits chosen to reduce impurity (Gini or entropy). Simple and interpretable, but prone to overfitting.
- **Random Forest [6]:** many decision trees trained on bootstrap samples with random feature subsets; their votes are averaged (bagging), which reduces variance.
- **SVM with RBF kernel [9]:** finds a maximum-margin boundary in a kernel-transformed space; inputs are standardised, and probability estimates are enabled for ROC curves.
- **XGBoost [7]:** gradient boosting that adds trees one at a time, each fitted to the gradient of the log-loss of the current ensemble, with shrinkage, row/column subsampling and L2 regularisation. The final score is the sum of the trees' outputs passed through a sigmoid.

3.3.2 Model Development and Hyper-parameter Tuning

Table 3.3 lists the hyper-parameter grids and the best configurations found by 5-fold cross-validation. Grids were kept small so that the whole pipeline runs in about seven minutes on a laptop CPU. All random seeds are fixed (42), so the results are reproducible.

Table 3.3: Hyper-parameter grids and best configurations (5-fold CV)

Model	Grid searched	Best (full)	Best (lite)
Decision Tree	criterion {gini, entropy}; max_depth {None, 10, 20}; min_samples_leaf {1, 2, 5}	criterion=entropy, max_depth=20, min_samples_leaf=1	criterion=entropy, max_depth=None, min_samples_leaf=1
Random Forest	n_estimators {200, 400}; max_depth {None, 20}; max_features {sqrt, 0.5}	max_depth=None, max_features=sqrt, n_estimators=200	max_depth=20, max_features=sqrt, n_estimators=400
SVM (RBF)	C {1, 10, 100}; gamma {scale, 0.1, 0.01}	C=10, gamma=0.1	C=10, gamma=0.1

Model	Grid searched	Best (full)	Best (lite)
XGBoost	n_estimators {200, 400}; max_depth {4, 6, 8}; learning_rate {0.05, 0.1}	learning_rate=0.1, max_depth=8, n_estimators=200	learning_rate=0.1, max_depth=8, n_estimators=400

3.3.3 Feature Importance Methods

- **Built-in importance:** impurity decrease (Decision Tree, Random Forest) and gain (XGBoost).
- **Permutation importance [11]:** the drop in test F1-score when one feature's values are shuffled (10 repeats). It works for every model, including the SVM.
- **SHAP [8]:** TreeExplainer computes each feature's contribution to each individual prediction; it is used for the summary plots and for the explanations returned by the API.
- **Ablation:** the best model is retrained on its top-k features (k = 5, 10, 15, 20, 30), ranked by SHAP on training data.
- **Full-vs-lite gap analysis:** the nine excluded features are added back to the lite set one at a time and cumulatively, measuring how much of the CV F1 gap each recovers.

3.3.4 Performance Metrics

The accuracy of the classifier is computed using Equation (3.1), and precision, recall and F1-score for the phishing class using Equations (3.2) to (3.4):

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \tag{3.1}$$

$$Precision = \frac{TP}{TP+FP} \tag{3.2}$$

$$Recall = \frac{TP}{TP+FN} \tag{3.3}$$

$$F1 = \frac{2\times Precision\times Recall}{Precision+Recall} \tag{3.4}$$

where TP, TN, FP and FN denote true positives, true negatives, false positives and false negatives respectively, with phishing as the positive class. ROC-AUC (the area under the receiver operating characteristic curve) measures how well the predicted probabilities rank phishing above legitimate websites across all thresholds. For stability, we report the mean and standard deviation of the 5-fold CV F1-score.

3.4 Tools and Technologies Used

- Programming language: Python 3.13; HTML, CSS and JavaScript for the web page and extension.
- Libraries / frameworks: scikit-learn 1.9.1, XGBoost 3.4.1, SHAP 0.53.0, pandas, NumPy, SciPy, Matplotlib, joblib; requests, BeautifulSoup4 and tldextract for feature extraction; FastAPI, Pydantic and Uvicorn for the API; pytest for testing; Chrome Extension Manifest V3; Vercel for hosting.
- Hardware: laptop with a 13th Gen Intel Core i9-13980HX CPU (24 cores, 32 threads) and 15.6 GB RAM; no GPU was used.

Chapter 4
Results and Discussion

This chapter reports the results of the experiments. All numbers are produced by the code in the GitHub repository and are read from its results/ folder.

4.1 Experimental Setup

All experiments ran on Windows 11 with Python 3.13 on the laptop described in Section 3.4, using the CPU only. The data was split once into 8,844 training and 2,211 test websites (stratified, seed 42). Hyper-parameters were tuned with GridSearchCV and 5-fold stratified cross-validation on the training set (scoring: F1 of the phishing class), with at most 8 parallel worker processes. The decision threshold is 0.5. Training time is the refit time of the best configuration; inference time is the median of 200 single-website predictions. The complete pipeline, including the importance analysis, took about seven minutes, and a second run from an empty folder reproduced every metric exactly.

4.2 Results

4.2.1 Model Comparison on the Full Feature Set

Table 4.1: Test-set performance on the full feature set (30 features, 2,211 test websites)

Model	Acc. (%)	Prec. (%)	Recall (%)	F1 (%)	ROC-AUC	CV F1 (mean ± std)	Train (s)	Pred. (ms)
Decision Tree	96.70	96.80	95.71	96.25	0.9780	0.9564 ± 0.0049	0.08	0.54
Random Forest	97.69	98.04	96.73	97.38	0.9964	0.9659 ± 0.0026	0.69	10.10
SVM (RBF)	97.56	97.73	96.73	97.23	0.9904	0.9595 ± 0.0032	5.89	0.77
XGBoost	97.60	97.64	96.94	97.29	0.9969	0.9649 ± 0.0019	0.20	2.33

Random Forest, SVM and XGBoost perform almost identically on the full feature set (Table 4.1): their accuracies lie within 0.14 percentage points, a difference of only one to three test websites. Random Forest has the highest accuracy (97.69%), while XGBoost has the highest phishing recall (96.94%), the highest ROC-AUC (0.9969) and the most stable

cross-validation score. The single Decision Tree is clearly weaker. Figures 4.1 and 4.2 show the ROC curves and confusion matrices.

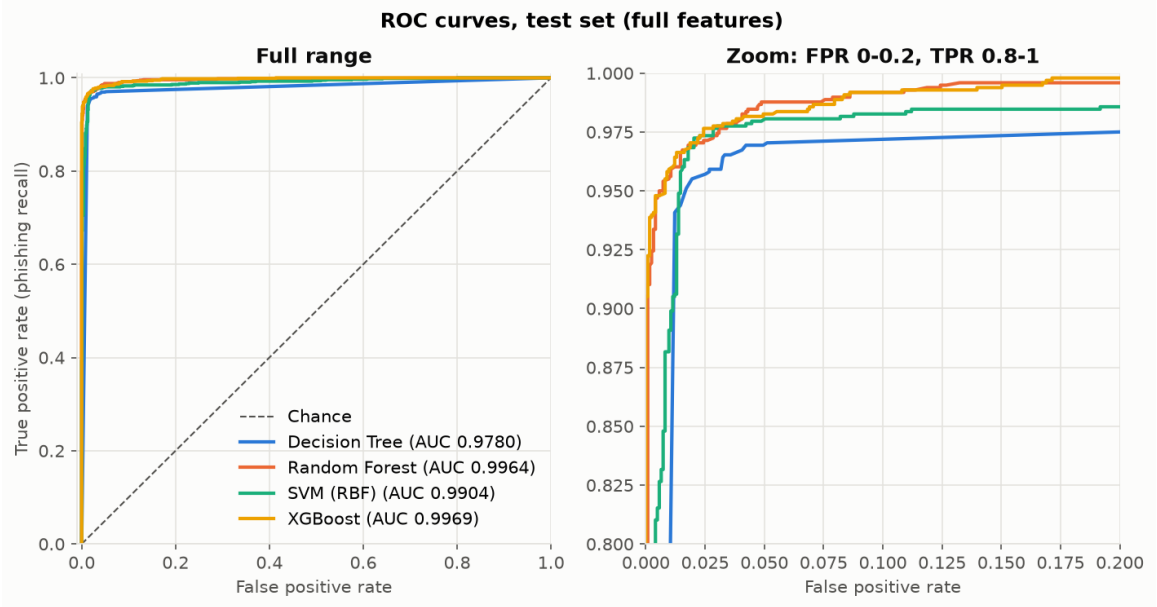


Figure 4.1: ROC curves on the test set (full feature set)

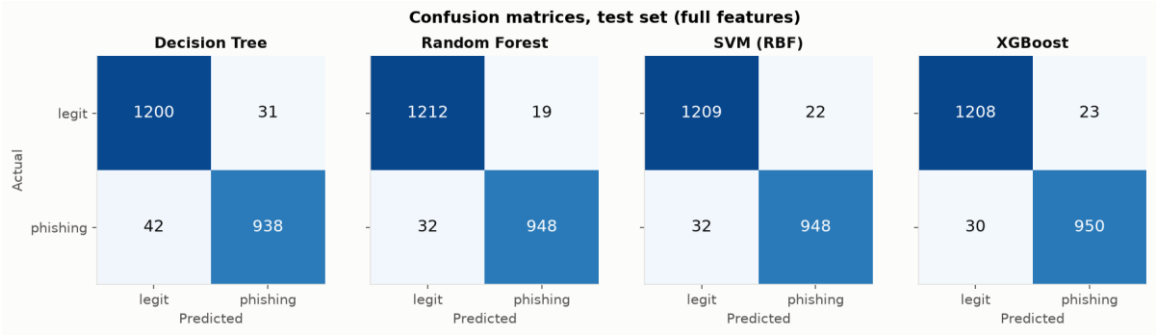


Figure 4.2: Confusion matrices on the test set (full feature set)

4.2.2 Model Comparison on the Lite (Deployable) Feature Set

Table 4.2: Test-set performance on the lite feature set (21 features)

Model	Acc. (%)	Prec. (%)	Recall (%)	F1 (%)	ROC-AUC	CV F1 (mean ± std)	Train (s)	Pred. (ms)
Decision Tree	94.53	93.43	94.29	93.85	0.9826	0.9392 ± 0.0030	0.02	0.89
Random Forest	94.80	94.45	93.78	94.11	0.9896	0.9438 ± 0.0031	1.71	16.73
SVM (RBF)	94.75	94.63	93.47	94.05	0.9793	0.9412 ± 0.0011	4.53	0.76
XGBoost	94.62	94.34	93.47	93.90	0.9905	0.9438 ± 0.0038	0.41	1.69

With only the 21 live-computable features, all models lose about three percentage points of accuracy (Table 4.2, Figure 4.3). Random Forest and XGBoost tie on CV F1-score (0.9438); XGBoost was selected for deployment because a single prediction takes 1.7 ms instead of 16.7 ms. The choice is made on cross-validation, not on the test set.

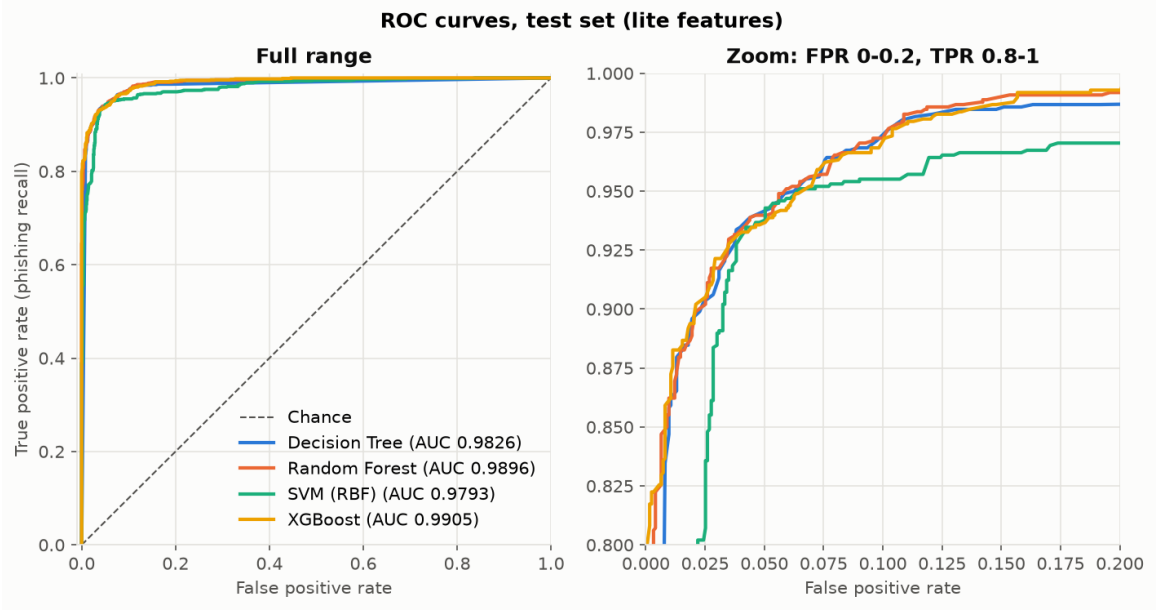


Figure 4.3: ROC curves on the test set (lite feature set)

4.2.3 Sensitivity to Duplicate Rows

To measure how much duplicates inflate the scores, each tuned configuration was retrained and re-tested after removing exact duplicates before splitting (Table 4.3).

Table 4.3: Accuracy and F1-score on the standard and de-duplicated splits

Features	Model	Train / test rows	Acc. standard (%)	Acc. dedup (%)	F1 standard (%)	F1 dedup (%)	F1 change
full	Decision Tree	4,679 / 1,170	96.70	92.65	96.25	92.79	-3.47
full	Random Forest	4,679 / 1,170	97.69	94.10	97.38	94.20	-3.18
full	SVM (RBF)	4,679 / 1,170	97.56	93.59	97.23	93.70	-3.53
full	XGBoost	4,679 / 1,170	97.60	94.87	97.29	94.99	-2.29
lite	Decision Tree	2,450 / 613	94.53	89.89	93.85	89.90	-3.95
lite	Random Forest	2,450 / 613	94.80	91.84	94.11	91.72	-2.39

Features	Model	Train / test rows	Acc. standard (%)	Acc. dedup (%)	F1 standard (%)	F1 dedup (%)	F1 change
lite	SVM (RBF)	2,450 / 613	94.75	91.19	94.05	90.97	-3.08
lite	XGBoost	2,450 / 613	94.62	91.52	93.90	91.36	-2.54

Every model loses 2.3 to 4.0 F1 points on unique data, so part of the headline accuracy comes from memorising repeated rows. XGBoost is the most robust: it has the best full-set accuracy on de-duplicated data (94.87%) and the smallest F1 drop.

4.2.4 Feature Importance

Permutation importance (Figure 4.4) and SHAP (Figure 4.5) agree that *sslfinal_state* (valid HTTPS) and *url_of_anchor* (links that point nowhere or to other domains) dominate every model, followed by *web_traffic*, *prefix_suffix*, *links_in_tags* and *having_sub_domain*. About ten features, such as *iframe*, *rightclick* and *port*, have almost no effect. Across all eight importance rankings the mean Spearman rank correlation is 0.82 (Figure 4.6); the only outlier is XGBoost's built-in gain importance, which is known to over-reward features that split small, pure groups of rows.

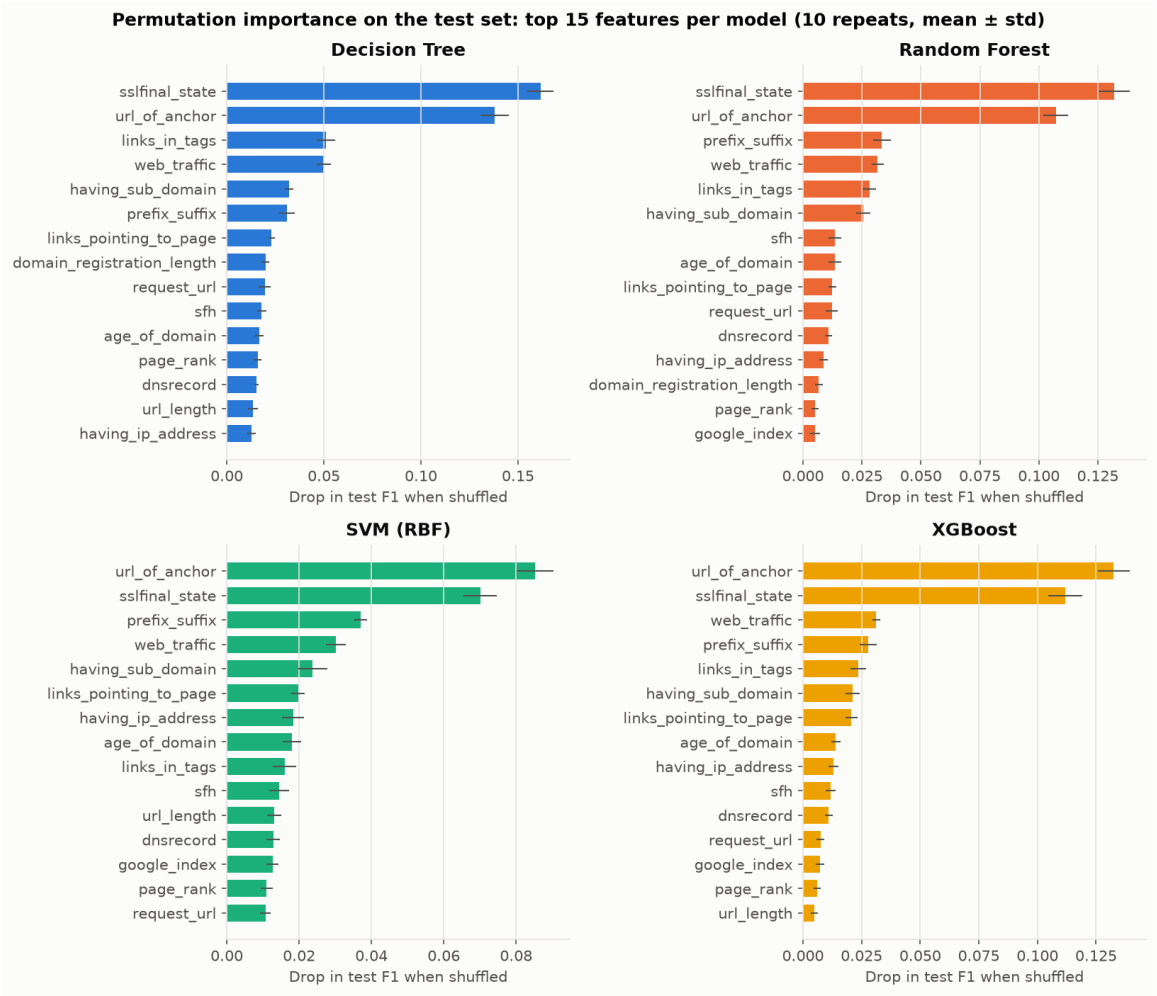


Figure 4.4: Permutation importance on the test set (top 15 features per model)

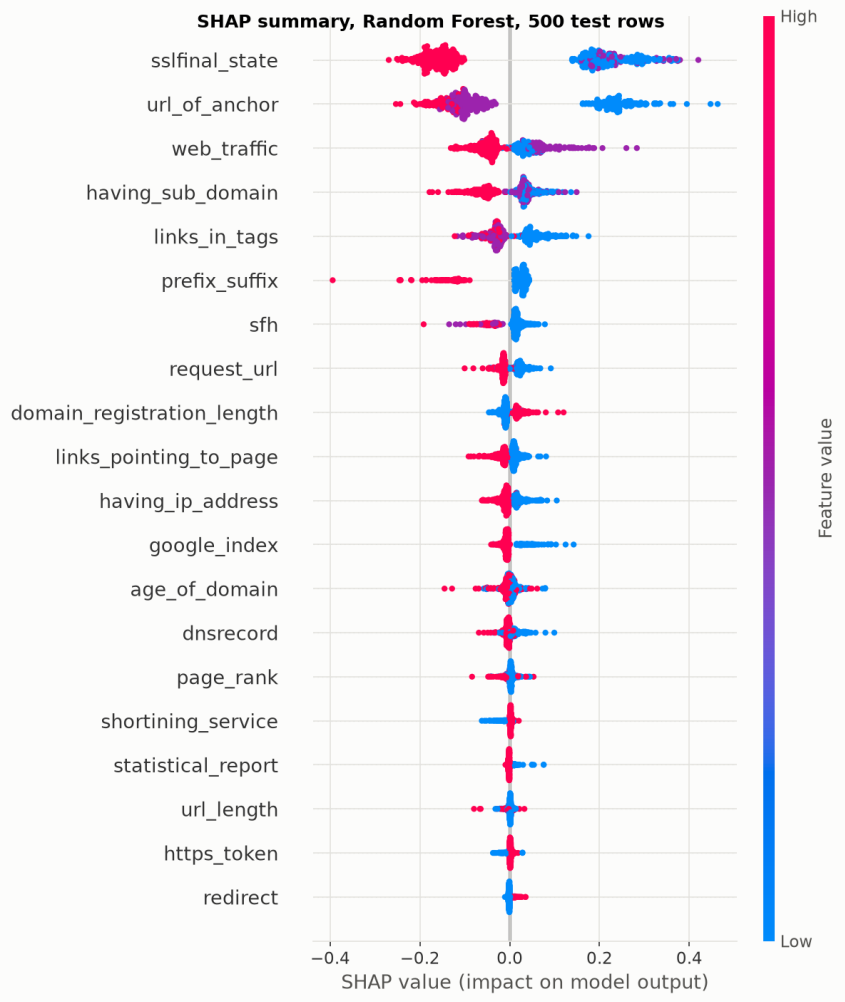


Figure 4.5: SHAP summary for the Random Forest (500 test websites)

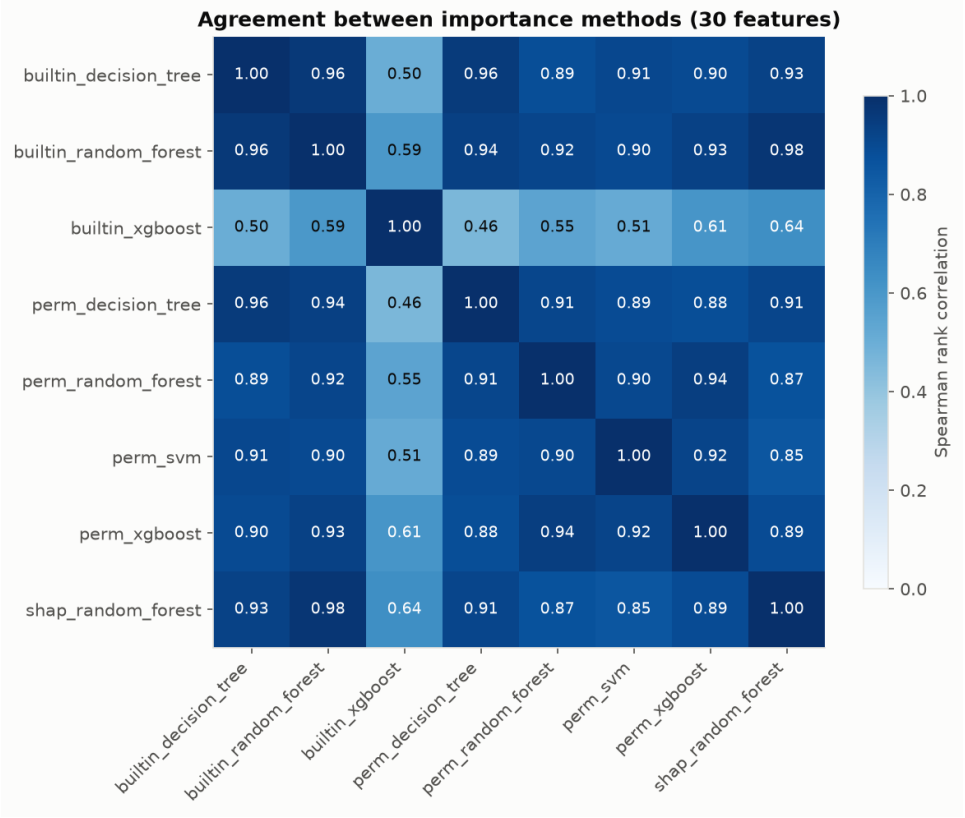


Figure 4.6: Agreement between importance methods (Spearman correlation)

The ablation in Figure 4.7 shows that five features already give 93% accuracy and fifteen features reach 96.97%, about 0.7 points below all thirty. Figure 4.8 explains the full-vs-lite gap: no single excluded feature recovers more than about a fifth of the gap (links_pointing_to_page 21%, web_traffic 18%), but together the excluded reputation features (backlinks, traffic rank, domain age, Google index, PageRank) recover all of it. The missing information is the site's reputation.

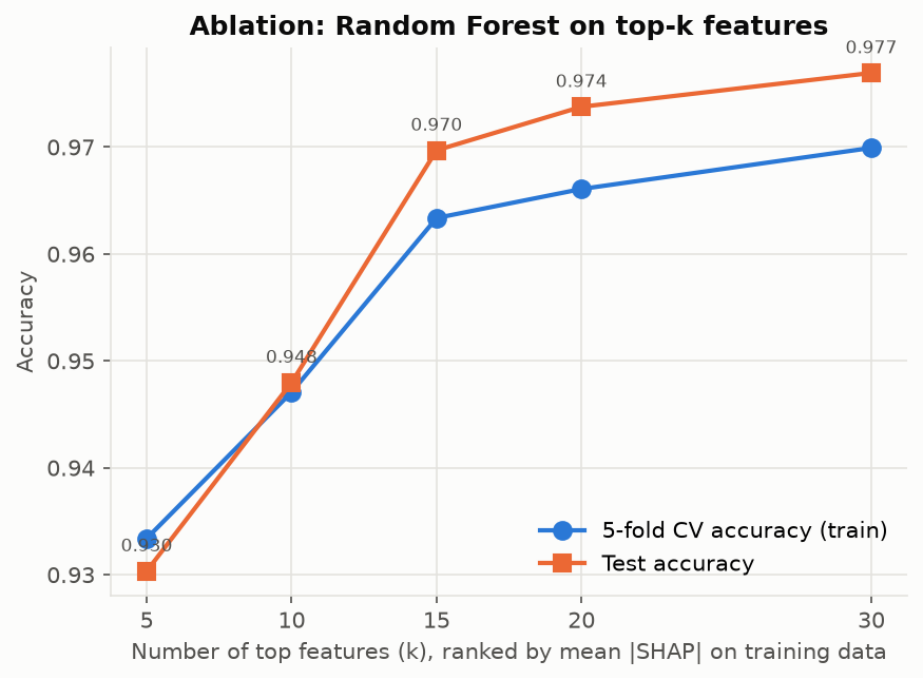


Figure 4.7: Ablation: Random Forest on its top-k features

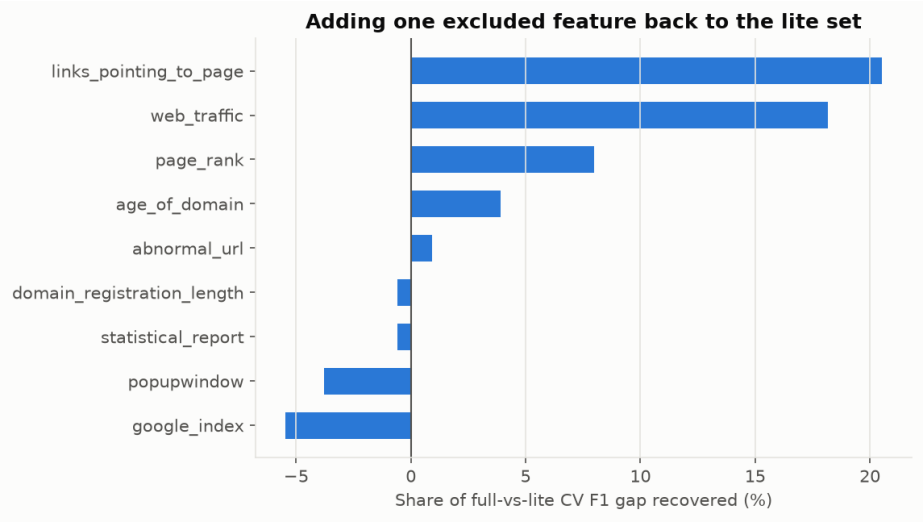


Figure 4.8: Share of the full-vs-lite gap recovered by each excluded feature

4.2.5 Sample Results of the Deployed Application

Figures 4.9 to 4.11 show the web application. A real legitimate site checked on the public deployment is classified as legitimate with its main reasons (Figure 4.9). A harmless fake phishing page built for testing (an "account verification" form that submits nowhere, links that go nowhere, a hidden iframe and disabled right-click) is classified as phishing with 99.5% probability (Figure 4.10). A modern, cleanly built fake page is **missed** (Figure 4.11): it looks technically clean, and this shows the main limitation of a model trained on 2015 data. In a check of 20 popular legitimate websites, none was flagged as phishing.

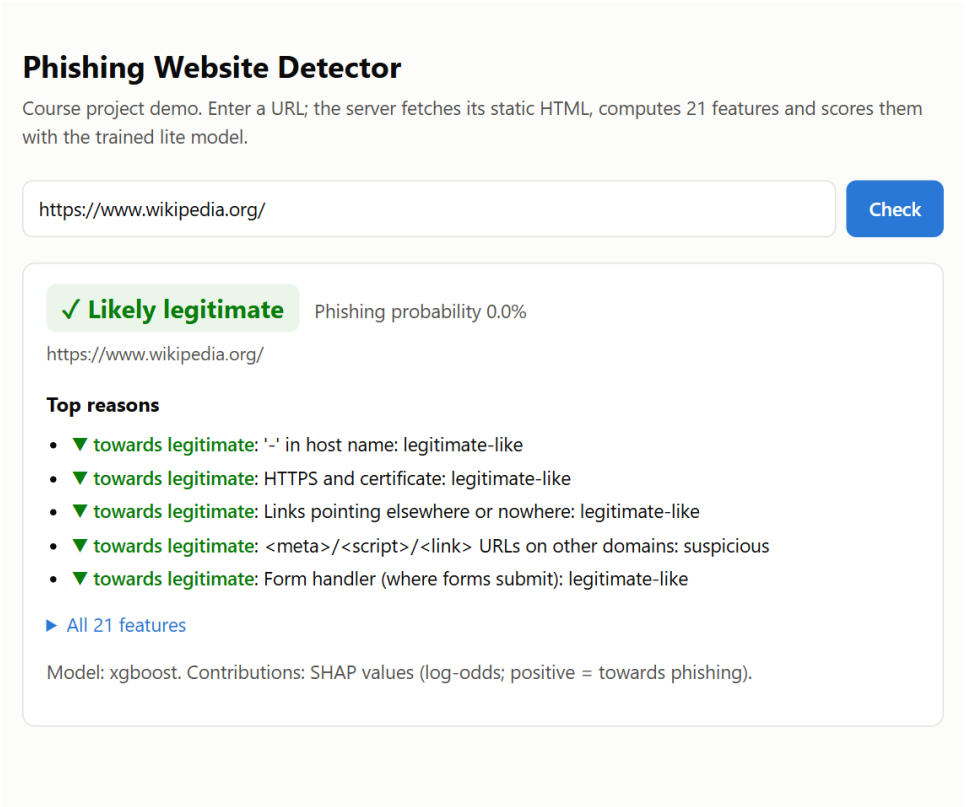


Figure 4.9: Sample output: a legitimate website checked on the public deployment

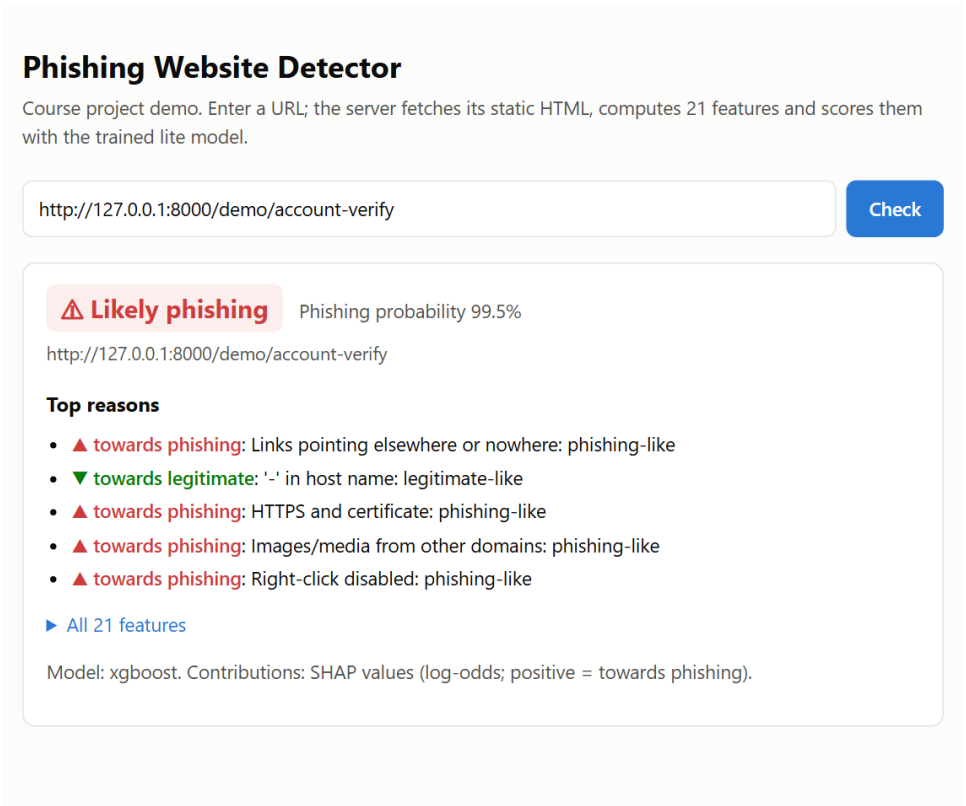


Figure 4.10: Sample output: the fake phishing test page is detected (99.5%)

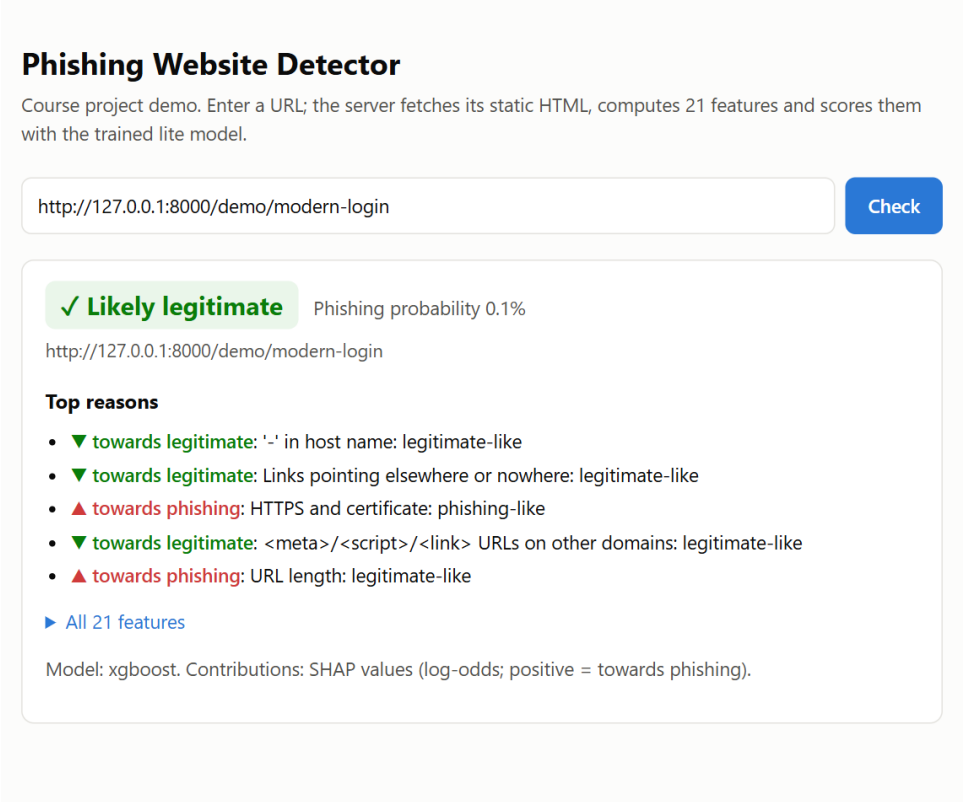


Figure 4.11: Sample output: a modern, clean fake phishing page is missed (limitation)

4.3 Comparison with Existing Methods

Table 4.4 compares the proposed method with existing methods on the same dataset. The numbers for existing methods are taken from the papers; they use different splits and protocols, so small differences should not be over-interpreted. Values reported with two decimals in the source papers are shown as percentages.

Table 4.4: Performance comparison of the proposed method with existing methods

Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
Extra Trees, Awasthi & Goel [3]	98.73	98	97	97
Tuned CNN, Almujahid et al. [4]	99	99	98	99
Stacking ensemble, Newaz et al. [5]	97.49	97.11	98.43	97.76
Proposed: Random Forest (full, 30 features)	97.69	98.04	96.73	97.38
Proposed: XGBoost (full, 30 features)	97.60	97.64	96.94	97.29
Proposed: XGBoost (lite, 21 live features, deployed)	94.62	94.34	93.47	93.90

On the full feature set, the proposed models are within about one point of the best published results, with simple, tuned models and a stricter protocol (a held-out test set used once). The project's main contributions lie elsewhere: it measures the effect of duplicate rows (which the compared studies do not), explains the predictions, quantifies the cost of using only deployable features, and delivers a working, explainable application.

4.4 Deployment Details

- **Feature extractor (src/extract.py):** computes the 21 lite features for a URL with the dataset authors' encoding rules: 9 from the URL string, a DNS lookup, a TLS certificate check and 10 from the page's static HTML (no JavaScript is executed). It uses 5-second timeouts and a 2 MB size limit; if the page cannot be fetched, the missing features are filled with their most common training value and the result is marked as partial.
- **Prediction API (FastAPI):** POST /predict returns the label, the phishing probability, all 21 features and the top five reasons (per-prediction SHAP values computed by XGBoost's built-in TreeSHAP); GET /health and a try-it web page are also provided. Inputs are validated (http/https only).
- **Chrome extension (Manifest V3):** a popup that sends the current tab's URL to the local API and shows the verdict and reasons. It requests only the activeTab permission and access to the local API.
- **Public deployment (Vercel):** a slim version of the API is hosted at https://phishing-website-detector-wheat.vercel.app. It refuses private and internal addresses (including through redirects) so that it cannot be used to probe the server's network, does not include the fake test pages, and limits each client to 20 checks per minute.
- **Testing:** 118 automated tests (pytest) cover every feature rule, the API, the public-mode safeguards and the saved models.

4.5 Link of GitHub Repository

The source code, trained models, results and documentation are available at: https://github.com/ParthKinage/Phishing-website-

The deployed web application is available at: https://phishing-website-detector-wheat.vercel.app

Chapter 5
Conclusion and Future Scope

5.1 Conclusion

This project built and evaluated an ML system for phishing website detection. On the UCI Phishing Websites dataset, Random Forest, SVM and XGBoost reach about 97.6% test accuracy, with Random Forest highest at 97.69% and XGBoost best on phishing recall and ROC-AUC; the Decision Tree is about one point lower. Using only the 21 features that can be computed live costs about three points (94.5-94.8%), and this gap is explained by the missing site-reputation features as a group. Built-in, permutation and SHAP importance agree that valid HTTPS and the destination of a page's links are the strongest signals. Removing the 47% duplicate rows lowers F1 by 2-4 points, which gives a more honest estimate of performance on unseen websites. The best lite model, XGBoost, was deployed as an explained web service with a web page, a Chrome extension and a public website, meeting all the objectives of Section 1.4.

5.2 Future Scope

- Train on a recent, larger dataset collected with the same extractor that is used in deployment, so that training and live features match exactly.
- Add modern reputation signals that can be computed today, such as domain age from RDAP, popularity from the Tranco list and certificate-transparency logs.
- Render pages in a headless browser to analyse JavaScript-built content, and add visual similarity to known brands to catch modern, cleanly built phishing kits.
- Evaluate the system on a live feed of verified phishing URLs, and combine the model with blocklists such as Google Safe Browsing.
- Publish the Chrome extension with a configurable API address and add user feedback for continual learning.

Chapter 6
List of Publications

6.1 List of Published Papers

No paper has been published from this work so far.

6.2 Details of Participation in Project Competitions

The project has not yet been presented at a project competition.

References

[1] R. M. Mohammad, F. Thabtah, and L. McCluskey, “Predicting phishing websites based on self-structuring neural network,” *Neural Computing and Applications*, vol. 25, no. 2, pp. 443–458, 2014.

[2] R. M. Mohammad, F. Thabtah, and L. McCluskey, “Phishing Websites,” UCI Machine Learning Repository, 2015. [Online]. Available: https://archive.ics.uci.edu/dataset/327/phishing+websites

[3] A. Awasthi and N. Goel, “Phishing website prediction using base and ensemble classifier techniques with cross-validation,” *Cybersecurity*, vol. 5, art. 22, 2022, doi: 10.1186/s42400-022-00126-9.

[4] N. F. Almujahid, M. A. Haq, and M. Alshehri, “Comparative evaluation of machine learning algorithms for phishing site detection,” *PeerJ Computer Science*, vol. 10, e2131, 2024, doi: 10.7717/peerj-cs.2131.

[5] A. Newaz, F. S. Haq, and N. Ahmed, “A sophisticated framework for the accurate detection of phishing websites,” arXiv preprint arXiv:2403.09735, 2024.

[6] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[7] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[8] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems 30*, 2017.

[9] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[10] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[11] A. Altmann, L. Toloşi, O. Sander, and T. Lengauer, “Permutation importance: a corrected feature importance measure,” *Bioinformatics*, vol. 26, no. 10, pp. 1340–1347, 2010.

Hosted website link: https://phishing-website-detector-wheat.vercel.app